

让 AI 更快地写， 让人更清楚地判断。

Coding SOP 的核心是把需求、实现与审查连成可验收的链路。先确定做什么，再让 Agent 并行实现，最后由人对关键取舍和交付结果负责。

双人工门控

垂直切片

每 Ticket 一个 Agent

TDD + Review

嵌入式教学案例

先固定方向 Spec 通过人工 Review，才进入完整流程的实现阶段。

再并行交付 按 Ticket 分工；依赖决定先后，Worktree 隔离改动。

最后交给判断 自动审查整理风险与证据，人工 CR 作最终决定。

00 / MAP

一条主链路，两种执行强度

工具承担局部职责，工程纪律把它们串起来。完整流程适合边界多、跨模块或需要多人并行的任务；边界清楚、验证成本低的小改可以走轻流程。

完整流程



橙色节点表示人工判断。图中名称表达工作职责，具体命令与参数取决于实际安装的工具。

输出逐级收敛：问题清单 → 审核通过的 Spec → 带依赖的 Ticket → 小范围实现与验证记录 → PR 证据包 → 人工结论。

不满足退出条件就回到相应阶段：验收不清回到对齐，接口约定改变回到 Spec，集成失败回到相关 Ticket。

轻流程



适用例：已有测试覆盖的错误码映射修正，输入输出明确，影响集中在一个文件。若已有明确授权，按已授权方案继续执行，无需重复确认。

升级条件：发现 ISR 与任务间共享状态、公共接口变更、协议兼容性影响或难以复现的时序问题时，扩大调查并重新评估流程强度。

流程可以分级，纪律不能分级。 小改也要先有简短方案与人的授权，改后运行适当测试。减少的是组织开销，不是对边界和结果的判断。

WHAT TO KEEP

把注意力留给真实约束

复用现有约定；仅给当前任务需要的上下文；保留可检验的验收条件。规则的价值在于帮助决策，而不是让每个任务携带更厚的指令。

WHAT HUMANS OWN

工具负责执行，人负责取舍

延迟、可用性、一致性、资源成本与可扩展性之间没有通用最优解。Agent 可以整理选项，人需要根据系统目标选择，并留下理由。

先把“看起来懂了”变成可检验的共识

`grill-me` 用连续追问暴露盲区；已有设计、协议或故障资料时，`grill-with-docs` 结合文档核对。此时的成果是问题边界与决策记录，还不是代码。

贯穿案例 · 教学假设

某 FreeRTOS 设备在 UART 连续突发输入时出现接收队列溢出和处理任务积压。当前目标是明确并实现可观测的丢弃策略，保护接收路径响应性；这不等价于承诺“永不丢包”。若真实产品要求无损，需重新评估背压、重传与吞吐预算，不能直接采用本例策略。队列容量、时序预算和负载阈值都必须由目标项目确认。

追问维度	需要弄清的事实	形成的约束 / 决策
现象与复现	队列真的满了，还是 DMA 覆盖、解析慢或日志阻塞？什么输入模式触发？	记录触发负载、持续时间、观测点；把推测根因与已有证据分开。
生产与消费	入队来自 ISR 还是任务？谁消费？队列中放字节、帧还是描述符？	明确执行上下文、数据所有权和对象生命周期，避免错误地套用线程 API。
满队列行为	丢新数据、丢旧数据、背压还是上报故障？协议能否重传？	教学方案选“丢新项并计数”；丢弃后是否破坏帧同步需要单独确认。
资源与时序	RAM 预算、ISR 时间预算、任务优先级和处理耗时分别是什么？	不能仅靠扩大队列掩盖持续输入速率高于消费速率的问题。
验证与非目标	如何观察溢出？怎样证明恢复？本次是否改调度或协议？	明确测试层级、必需硬件证据和不做事项，防止修复范围不断扩张。

对齐阶段的退出条件

- 业务目标、输入输出与影响范围能被复述。
- 关键边界已有决策；未决项标明影响、负责人和处理方式。
- 验收可观察，必要信息足以写出 Spec。

遇到资料冲突怎么办

文档、代码和日志可以互相矛盾。把冲突写成待验证项：例如“文档称单生产者，但代码存在第二条入队路径”。在确认前，不把单生产者当作实现前提。

第一道人工门控：批准问题定义与实现边界

to-spec 把对齐结果整理成设计约定。人工 Review 关注“这是不是值得实现的正确方案”，而不只是检查文档是否完整。

Spec 示例 · UART 满队列可观测处理

教学草案 · 待项目确认

目标	队列满时快速返回，保留已接收的数据顺序，并提供可解释的丢弃计数。消费恢复后，接收路径可继续工作。
行为契约	成功入队的项按约定顺序交给消费任务；满队列时丢弃新项并计数。帧失步恢复由协议层既有机制处理；若该机制缺失，必须补充设计。
不变量	ISR 不阻塞、不动态分配、不做格式化日志；队列操作符合执行上下文；跨上下文统计读取不得产生撕裂或未定义竞争。
非目标	本轮不改变 UART 协议、不重排全部任务优先级、不引入通用异步框架。若验收需要这些改动，先更新范围再实施。
验收 A1	在容量为 N 的测试队列中，先填入 N 个不同标识的项，再入队 1 个；新项被拒绝，计数增加 1，已有项内容与顺序不变。
验收 A2	消费一个项后再提交新项，入队成功；排空后的结果与预期序列一致，证明溢出后路径可恢复。
验收 A3	目标板突发负载测试记录 ISR 时间、丢弃计数、队列占用和消费进度；阈值由项目预算填写，未测量前不得标记通过。
验证边界	Host 测试验证策略；目标构建检查平台适配；板级测量验证真实时序与中断行为。测试替身不能替代目标硬件证据。
风险与回退	丢新项可能丢失重要协议内容，先确认上层容错；计数溢出策略和复位时机写入约定。回退须恢复旧行为并保留变更记录。

人要看清三个问题

- 对不对**：是否解决已确认的问题，是否误把假设当事实？
- 值不值**：丢弃策略与资源代价是否符合设备目标？
- 验不验得出**：如何得到失败与成功的证据，谁完成板级验证？

批准之后仍可修订

Spec 是当前共识，不是冻结一切。实现时若发现第二个生产者、队列项生命周期不符或预算无法满足，应先说明影响并更新相关决定，再恢复依赖任务。

门控结果应可追踪：批准、需修改或暂缓，并附理由。仍影响核心行为的未决问题，应在进入实现前解决；已经明确批准的范围无需重复走形式化授权。

03 / SLICE

按可验收行为切片，用依赖图决定并行

to-tickets 把 Spec 拆成足够小、可以单独执行与验收的 Ticket。垂直切片围绕完整行为贯穿必要模块，避免只拆成“写驱动 / 写测试 / 写文档”后全部等待最后联调。

Ticket	独立交付的行为与证据	依赖	责任边界
T1 接收基线	建立正常接收行为的可复现测试，锁定现有队列适配接口与输入序列。	无	基线测试与队列适配契约
T2 满队列策略	完成丢新项行为、计数与恢复测试，覆盖 A1 / A2。	T1	策略逻辑与对应测试
T3 占用观测	通过独立诊断入口安全读取队列占用 / 高水位，并有正常负载下的观测测试。	T1	诊断入口与统计快照测试
T4 集成压力验证	合并策略与观测，在目标负载下验证恢复、时序和统计一致性，形成 A3 记录。	T2、T3	压力场景与集成证据



DAG 是有向无环依赖图。它解决先后顺序；文件所有权与接口约定解决并行协作。T3 若需要修改 T2 的公共状态，应先拆出共享约定或改为串行。

试一遍：依赖满足才允许开工

每张卡依次经历“执行 → 待审 → 完成”。这是流程演示，按钮不会运行真实代码或测试。

T1 · 接收基线

可执行

固定正常接收行为与队列接口，为后续切片提供稳定输入。

依赖：无

T2 · 满队列策略

等待依赖

交付丢新项、计数与恢复测试，覆盖 A1 / A2。

依赖：T1

T3 · 占用观测

等待依赖

交付安全诊断快照及相应测试，可与 T2 并行。

依赖：T1

T4 · 集成压力验证

等待依赖

同时消费策略与观测产物，形成目标板验证记录。

依赖：T2、T3

已重置。先完成 T1，T2 与 T3 才可并行。

一个可执行 Ticket 要带什么

- 目标行为与对应的 Spec 验收编号。
- 依赖、输入、修改范围和禁止越界事项。
- 测试方法、交付物与完成条件。
- 发现接口冲突、需求变化或验证阻塞时的上报路径。

依赖完成，不等于“代码写完”

下游需要的是可用的接口与被接受的产物。上游验证未过或审查尚有阻塞意见时，不把任务标成完成。需要消费上游代码的 Worktree，还必须显式同步已确认的提交。

04 / IMPLEMENT

一个 Ticket，一个 Agent，一份隔离工作区

implement 负责把小任务落成代码。Subagent 获得该 Ticket 所需的干净上下文，Worktree 隔离分支和文件改动；任务内部执行 TDD 与 code-review。

分派给 Agent 的最小上下文

相关 Spec、当前 Ticket、项目规则、现有接口、必要文件与测试入口。明确文件责任，禁止回滚其他人的改动；公共接口变化必须协调。

“上下文干净”指减少与任务无关的会话历史，不是省略必要的设计约束。

Worktree 能隔离什么

它把各 Ticket 的工作目录与分支分开，减少互相覆盖。它不会自动消除逻辑冲突，也不会隔离共享数据库、测试设备、串口或外部构建缓存。

教学实践：板卡与串口按资源锁串行使用；共享接口先约定，集成后再次验证。

RED · 先看正确失败

围绕验收写最小失败测试，确认失败源于目标行为缺失。

GREEN · 最小实现

实现恰好满足约定的行为，运行相关测试并保存结果。

REFACTOR · 保持通过

必要时整理结构、复用现有逻辑，再运行受影响测试。

REVIEW · 回到证据

检查边界与并发；有问题则补测试、修复，再审查。

伪代码 A · 让失败约束满队列行为

```
# 教学伪代码：队列容量与 API 为示意，需替换为项目测试设施。
# 通过可区分的项检验“丢新项目且保留旧顺序”，避免只检查返回值。
test overflow_preserves_accepted_items():
    queue = queue_with_capacity(2)          # 容量 2 只用于构造最小边界。
    stats = fresh_stats()                   # 每次测试使用独立统计，避免其他用例影响断言。
    receiver = receiver_using(queue, stats)
    receiver.accept(A)
    receiver.accept(B)
    result = receiver.accept(C)              # 第三项必须触发既定的丢弃策略。

    assert result == DROPPED
    assert stats.dropped_delta() == 1        # 统计变化必须与一次拒绝一致。
    assert queue.consume() == A
    assert receiver.accept(D) == OK          # 释放空间后应恢复，不能永久拒收。
    assert queue.drain() == [B, D]           # 旧数据顺序与恢复后的新数据都要保留。

# RED 阶段应因策略缺失而失败；编译错误或测试设施损坏先单独修复。
# 用失败定位目标行为，再写最小实现；通过后增加项目需要的其他边界。
```

伪代码 B · ISR 路径里的非阻塞满队列处理

```
# 教学伪代码：不对应可直接调用的 RTOS API。
# 前提：队列项所有权已约定，队列操作支持 ISR，统计原语在目标平台安全。
on_uart_item_from_isr(item):
    outcome = queue.try_send_from_isr(item) # 使用一次原子尝试，避免先查满再写的竞态。

    if outcome == FULL:
        stats.increment_drop_from_isr()      # 使用经确认的并发与溢出策略；不能阻塞。
        return DROPPED                      # 丢新项，保持已经接收项的顺序与所有权。

    if outcome != OK:
        fault.record_from_isr(outcome)      # 满队列以外的错误单独记录，不能伪装成普通丢弃。
        return ERROR                       # 故障处理同样必须满足中断上下文限制。

    scheduler.request_yield_if_needed()     # 是否切换由 RTOS 约定与唤醒结果决定。
    return OK

# 任务侧只通过安全快照读取统计，禁止无保护读取可能撕裂的宽计数器。
# 真实 ISR 时间、统计读写和调度行为必须在目标平台验证。
```

TDD 的证据边界：Host 测试可以证明策略与序列，无法单独证明真实中断延迟、DMA 一致性或任务调度。无法在本轮获得板级数据时，应记录“待验证”及责任人，不把构建通过写成硬件验证通过。

05 / KEEP IT SMALL

短 Diff 的价值，是降低理解与审查成本

ponytail 对新增内容提出必要性检查：这段逻辑是否需要存在？项目是否已有能力？现有库能否完成？是否能用更直接的表达解决？

删掉没有贡献的结构

- 为一个不会扩展的实现预先建立多层接口。
- 把已有常量再包装成语义相同的新常量。
- 复制项目已经具备的适配和错误处理能力。
- 只复述语句动作的过程性注释。

保留真正承担约束的内容

- 输入来自不可信边界时必要的参数检查。
- 中断与任务之间的同步、生命周期约定。
- 解释丢弃策略、溢出规则和故障路径的注释。
- 证明行为与边界的测试和诊断证据。

代码更短不是独立验收目标。 嵌入式中的屏障、临界区、硬件异常保护可能看起来“啰嗦”，却承担必要职责。删减前要回答它保护了什么，并用证据确认仍然成立。

浅层目录示意 · 用文件责任帮助人审查

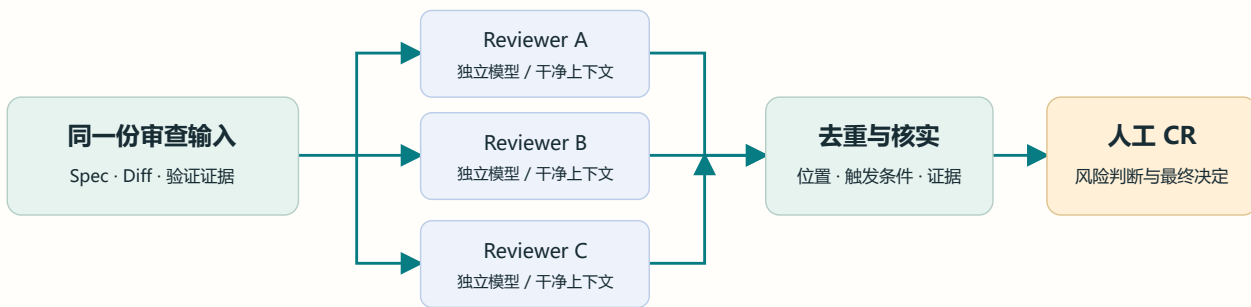
```
project/
├─ docs/
│   ├── uart-rx-spec.md          # 保存范围、行为约定、验收与人工决策。
│   └─ uart-rx-tickets.md       # 保存切片、依赖、责任人与状态。
├─ src/
│   ├── uart_rx_policy.c        # 仅处理已约定的接收与满队列策略。
│   └─ uart_rx_diagnostics.c    # 提供符合并发约束的统计与诊断入口。
└─ tests/
    ├── uart_rx_policy_test.c    # 覆盖拒绝、顺序保持和恢复边界。
    └─ uart_rx_stress.md        # 记录板级输入、测量方式、结果与限制。
```

教学目录：优先适配项目现有结构，不要求为了流程新建这些文件。

从 Ticket 自洽，走到 PR 整体正确

Ticket 内 review 解决局部实现问题；PR 级 review 检查合并后的整体行为。多模型在干净上下文中交叉审查，可以补充不同视角，但最终结论仍由人判断。

层级	重点问题	应交给下一层的证据
Ticket 内部	行为是否满足验收？错误路径、共享状态与测试覆盖是否合理？	小范围 Diff、测试结果、已修复问题与剩余限制。
PR 整体	多个切片组合后接口一致吗？统计是否重复？构建配置、资源预算与集成场景是否遗漏？	需求到代码 / 测试的映射、集成结果、去重后的风险清单。
人工 CR	关键取舍能否接受？证据是否足够？模型意见是否有依据？还需补哪些验证？	批准、退回修改或等待证据的明确结论，以及剩余风险的责任安排。



图中 3 路仅为结构示意，实际数量按任务和审查能力选择。同一套审查要求有助于比较结论；多票一致也不能替代可复现证据。

把模型意见变成可判断的信息

每条意见至少包含位置、触发条件、影响、证据与建议。重复意见合并，互相矛盾的意见保留争议点。无法确认的猜测标成待验证，不能包装为确定缺陷。

UART 案例中的高价值审查点

满队列失败后是否误释放已有缓冲区？任务侧读取计数是否安全？诊断日志是否拖慢接收？T2 与 T3 是否各自维护一份不一致的状态？板级测量是否覆盖峰值负载？

▼ PR 证据包：让人快速建立全局判断

一页说明：修改前的可复现问题、修改后的行为、主要取舍；**验收映射：**A1 / A2 / A3 对应哪项验证、当前状态；**验证记录：**环境、版本、输入、结果与日志位置；**审查结论：**已解决、待核实、接受的风险；**回退方式：**如何恢复旧行为，以及回退本身的影响。

07 / CAPACITY

并发上限，取决于人能消化多少审查

增加 Agent 和 Worktree 会提高产出速度，但人仍要理解方案、核对证据并处理争议。把任务启动速度限制在审查可承受范围内，才能避免大量“写完但无人确认”的积压。

演示：每轮产出超过审查能力会怎样

简化教学模型：假设每个 Agent 每轮交付 1 个同等大小 Ticket，不考虑依赖与返工。数值不代表真实团队能力。

并行 Agent 数

4

每轮可完成审查的 Ticket 数

2

预计每轮新增待审积压

2 个 Ticket

生产速度超过审查速度。先减少并发、缩小切片或提高证据质量，再增加新任务。

计算： $\max(0, \text{本轮产出} - \text{本轮审查完成量})$ 。实际管理还要观察返工率、风险等级与任务大小。

减少审查输入

让 Diff 聚焦一件事，删除不必要代码，将机械检查交给工具，把人注意力留给取舍与风险。

提高证据可读性

用验收映射、明确复现条件和去重结论帮助人快速判断，避免把多份长报告原样堆过去。

控制在制任务

待审任务持续积压时暂停新开工，优先完成审查和返修。并发数随团队能力调整。

08 / USE IT

从下一次任务开始，留下可复用的交付证据

先选流程强度，再按阶段判断是否达到退出条件。下面是可直接改写的分派模板与交付自检；它们是教学实践补充，需适配项目规则。

```
ticket: T2 满队列丢新项策略
spec_acceptance: [A1, A2]      # 用已批准的验收限制任务范围。
depends_on: [T1]                 # 上游接口与测试基线被接受后才能开始。
owner: one_subagent             # 每个 Ticket 由一个明确责任主体交付。
workspace: isolated_worktree    # 与其他任务分离；共享外设仍需协调。
scope:
  - 接收策略与对应测试          # 具体文件以项目结构为准，避免无关重构。
constraints:
  - ISR 非阻塞、无动态分配
  - 沿用队列适配契约            # 公共接口有变化时先协调，不能单边推进。
  - 不回滚其他人的改动
verification:
  - 先见到 A1/A2 的目标失败     # 测试设施失败不能当作有效 RED 证据。
  - 最小实现后运行相关测试
  - code-review 并处理阻塞项
handoff:
  - Diff、测试结果、剩余风险    # 板级未测项目明确写待验证，禁止推断为通过。
on_blocked:
  - 报告事实、影响与所需决策    # 缺信息或接口冲突时停止相关实现，避免扩大偏差。
```

交付前自检

勾选仅保存在当前页面会话中，用于自查；不会执行测试，也不会写入项目状态。

☐ 范围与流程强度明确；必要的方案或 Spec 已获授权。	☐ Ticket 有可验收行为、依赖、责任与完成条件。
☐ 并行改动隔离；共享接口和外设资源已协调。	☐ 相关测试已运行；结果、环境与限制有记录。
☐ Diff 保持聚焦；注释解释意图、约束和失败处理。	☐ Ticket 与 PR 两层审查完成，阻塞问题已处理。
☐ 硬件或集成待验证项被明确标记，未冒充通过。	☐ 人工 CR 有明确结论；剩余风险有责任安排。

真正稳定的能力：理解系统，识别边界，判断风险，选择取舍。工具可以替换，流程可以精简；每一步都应让下一步拥有更可靠的信息。

